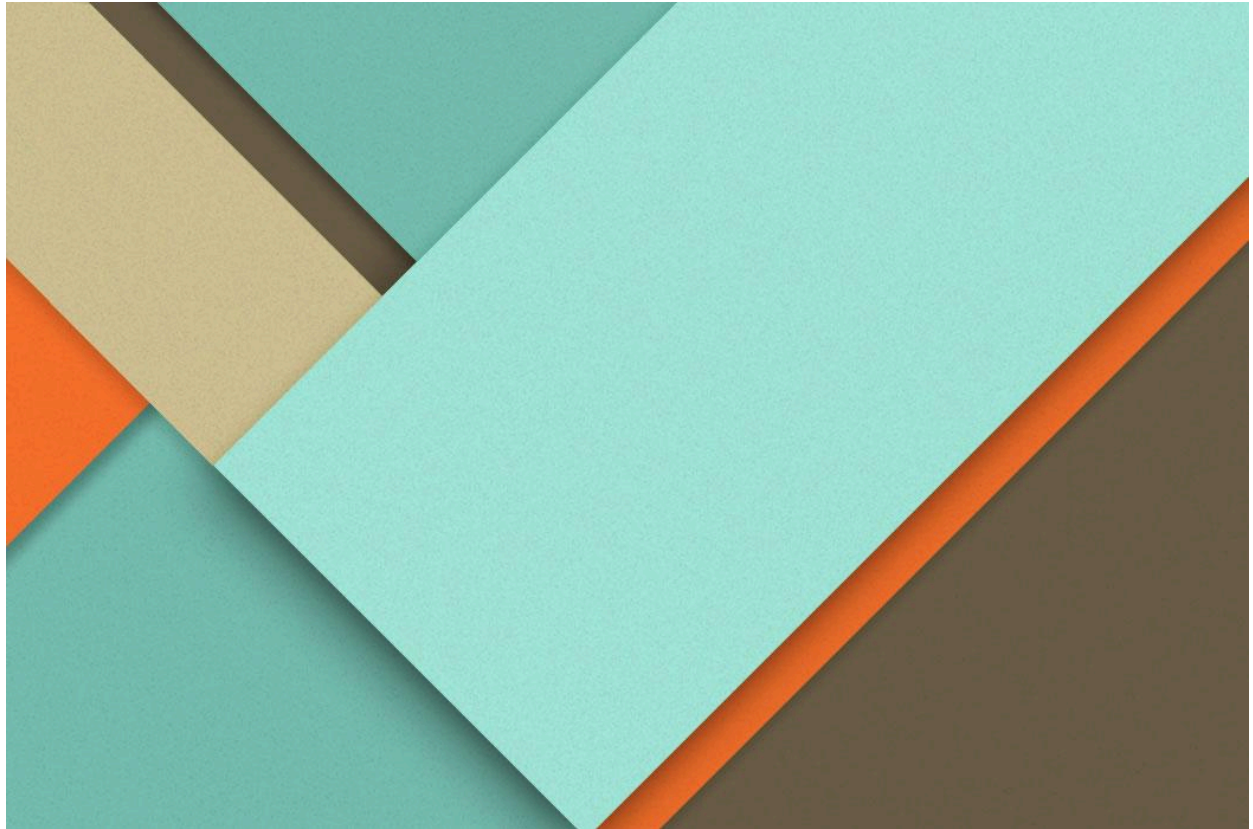




Videothek

Modul 295 - Backend für Applikationen realisieren

Yoshi Gyger
20.04.2026
24-295-F

Projektbeschreibung (Idee)

Ich habe mir vorgenommen, das Backend einer Videothek zu erstellen. Im Admin Dashboard sollte man Filme, wie Serien hochladen und verwalten können. Der User kann diese dann bewerten und ein Review schreiben. Nutzer können durch Filter einen Film oder eine Serie finden.

Der Admin Dashboard ist mit einem Login geschützt. Um Spam zu verhindern, müssen User auch einen Account haben, um die Bewertung und Review Funktion zu nutzen.

Ziel

Das Ziel des Projekts ist, ein Backend für eine Videothek zu erstellen. Es soll mir klar machen, wie ich dies selber umsetzen kann in einem gewissen Zeitrahmen.

Es sollte für den Admin möglich sein, Filme/Serien hinzuzufügen, zu bearbeiten und löschen zu können.

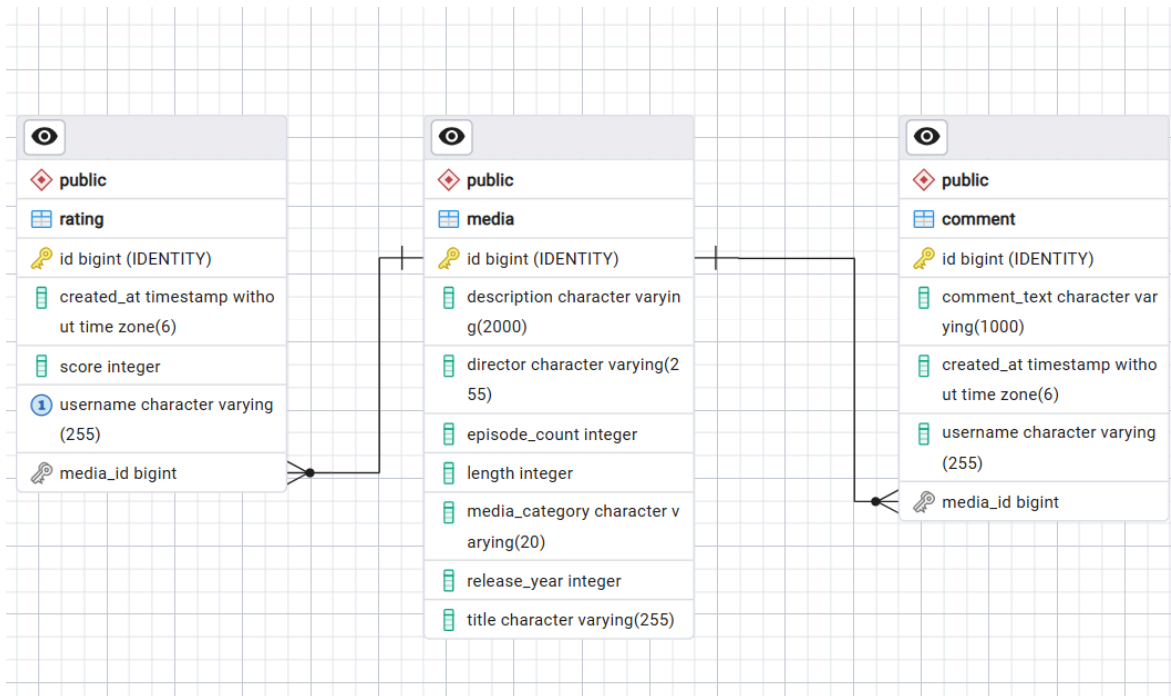
Die Applikation sollte eine Kommentar- und Bewertungsfunktion für Nutzer haben. Diese sollten nur von den erstellten User bearbeitet und gelöscht werden können.

Technologie-Stack

Technologie	Version	Zweck
Java	26	Programmiersprache
Spring Boot	4.0.5	Backend-Framework
Spring Security	7.0.4	Authentifizierung & Autorisierung
Keycloak	—	Identity Provider (OAuth2/JWT)
Hibernate / JPA	7.2.7	Datenbankzugriff
PostgreSQL	—	Datenbank
Springdoc OpenAPI	3.0.1	API-Dokumentation (Swagger UI)
JUnit	6.0.3	Testing
Mockito	5.20.0	Unit-Test Mocking
Lombok	1.18.42	Boilerplate-Reduktion

ERM

Das Projekt hat drei Tabellen: media, rating und comment. Die comment- und rating-Tabelle haben einen Foreignkey von media.



Sicherheit & Rollen

Die API verwendet JWT-Bearer-Tokens von Keycloak. Es gibt drei Rollen:

Rolle	Berechtigungen
ROLE_read	GET-Endpoints, Kommentare schreiben, Bewertungen abgeben
ROLE_update	Eigene Kommentare bearbeiten
ROLE_admin	Alle CRUD-Operationen, Admin-Endpoints

Users und die entsprechenden Rollen:

User	Rollen	Beschreibung
admin	ROLE_read, ROLE_update, ROLE_admin	Admin hat alle Berechtigungen.
user	ROLE_read, ROLE_update	User können bestimmte Sachen lesen und updaten.
yoshi	ROLE_read, ROLE_update	Yoshi ist ein Test-User für das Überprüfen von Kommentar, Berechtigungen und Rating AVG.

Endpunkte

Media /api/media

Methode	Pfad	Rolle	HTTP Status	Beschreibung
GET	/api/media	Read	200	Alle Medien abrufen
GET	/api/media/{id}	Read	200 / 404	Medium per ID abrufen
POST	/api/media	Admin	201 / 400	Neues Medium erstellen
PUT	/api/media/{id}	Admin	200 / 404	Medium aktualisieren
DELETE	/api/media/{id}	Admin	200 / 404	Medium löschen

Comments /api/media/{mediaId}/comments

Methode	Pfad	Rolle	HTTP Status	Beschreibung
GET	/api/media/{mediaId}/comments	Read	200 / 404	Kommentare eines Mediums
POST	/api/media/{mediaId}/comments	Read	201 / 404	Kommentar hinzufügen
PUT	/api/comments/{id}	Update	200 / 403 / 404	Eigenen Kommentar bearbeiten
DELETE	/api/comments/{id}	Admin	200 / 403 / 404	Kommentar löschen

Ratings /api/media/{mediaId}/ratings

Methode	Pfad	Rolle	HTTP Status	Beschreibung
POST	/api/media/{mediaId}/ratings	Read / Admin	201 / 409 / 404	Bewertung abgeben (1–10, einmalig pro User)
GET	/api/media/{mediaId}/ratings/average	Read	200 / 404	Durchschnittsbewertung abrufen

Admin `/api/admin`

Methode	Pfad	Rolle	HTTP Status	Beschreibung
GET	<code>/api/admin/media</code>	Admin	200 / 403	Alle Medien (Admin-View)
GET	<code>/api/admin/comments</code>	Admin	200 / 403	Alle Kommentare
DELETE	<code>/api/admin/comments/{id}</code>	Admin	200 / 404 / 403	Kommentar ohne User-Check löschen
GET	<code>/api/admin/stats</code>	Admin	200 / 403	Dashboard-Statistiken

Fehlerbehandlung

HTTP Status	Bedeutung	Beispiel
200	Erfolg	Ressource zurückgegeben oder gelöscht
201	Erstellt	Neues Medium / Kommentar / Bewertung erstellt
400	Ungültige Eingabe	Validierungsfehler bei @Valid
403	Zugriff verweigert	Falscher User oder fehlende Rolle
404	Nicht gefunden	Medium / Kommentar mit ID nicht vorhanden
409	Konflikt	User hat dieses Medium bereits bewertet

Tests

Controller-Test (MediaControllerTests)

Test	Methode	Beschreibung
<code>testGetMedia()</code>	GET <code>/api/media</code>	Prüft ob Medien zurückgegeben werden
<code>testSaveMedia()</code>	POST <code>/api/media</code>	Erstellt ein neues Medium
<code>testUpdateMedia()</code>	PUT <code>/api/media/{id}</code>	Aktualisiert ein Medium
<code>testDeleteMedia()</code>	DELETE <code>/api/media/{id}</code>	Löscht ein Medium und prüft via Repository

Repository-Tests(MediaRepositoryTests)

Test	Methode	Beschreibung
<code>testGetMedia()</code>	GET <code>/api/media</code>	Prüft ob Medien zurückgegeben werden
<code>testSaveMedia()</code>	POST <code>/api/media</code>	Erstellt ein neues Medium
<code>testUpdateMedia()</code>	PUT <code>/api/media/{id}</code>	Aktualisiert ein Medium
<code>testDeleteMedia()</code>	DELETE <code>/api/media/{id}</code>	Löscht ein Medium und prüft via Repository

Service-Tests(MediaServiceTests)

Test	Beschreibung
<code>createMedia()</code>	Prüft ob <code>save()</code> auf dem Repository aufgerufen wird
<code>findMedia()</code>	Prüft ob <code>findById()</code> auf dem Repository aufgerufen wird
<code>deleteMedia()</code>	Prüft ob <code>deleteById()</code> auf dem Repository aufgerufen wird

Swagger UI

Die interaktive API-Dokumentation ist erreichbar unter:

<http://localhost:9090/swagger-ui.html>